

Project 3 Writeups

- PCA

- Qpca1 Implement PCA:

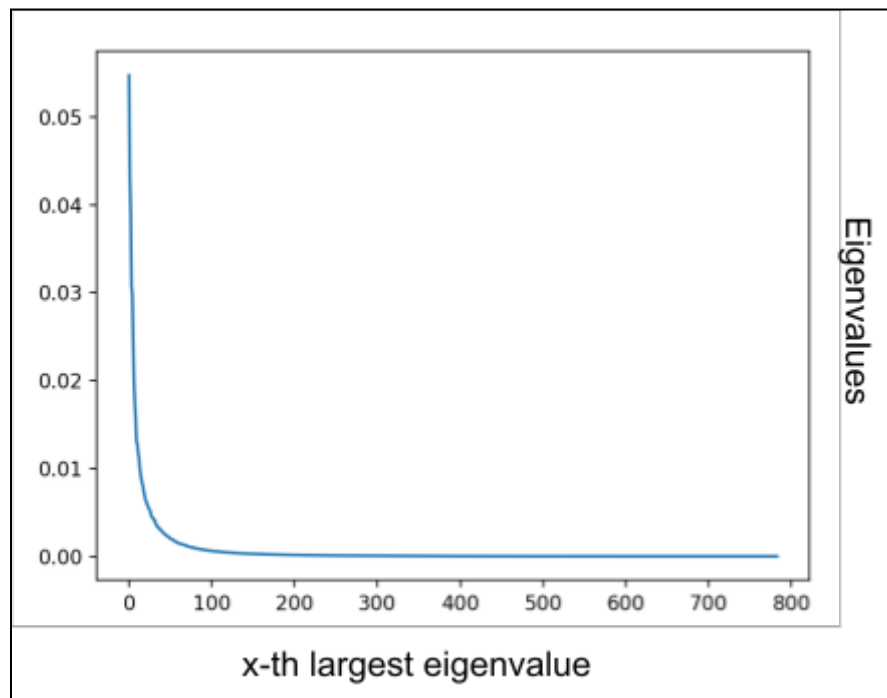
- Done in dr.py.

- Qpca2 Plot the normalized eigenvalues (include the plot in your writeup). How many eigenvectors do you have to include before you've accounted for 90% of the variance? 95%? (Hint: see function `cumsum`):

```

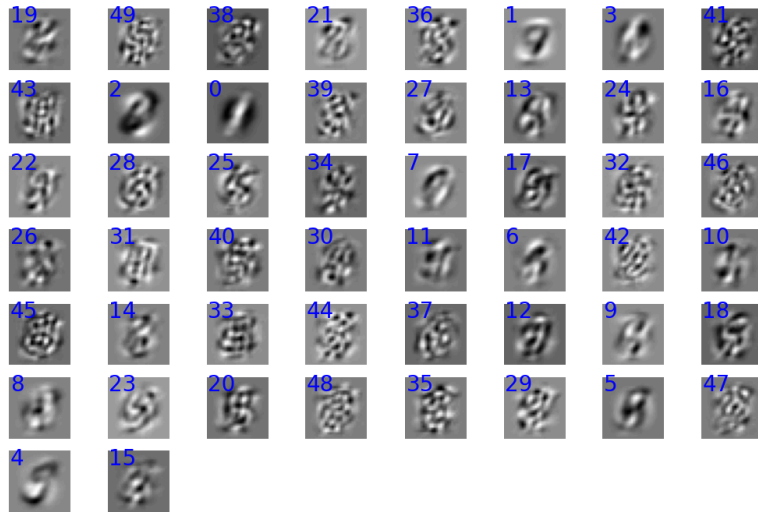
10 # QPCA 2
11 (X, Y) = datasets.loadDigits()
12 (P, Z, evals) = dr.pca(X, 784)
13 plot(list(range(len(evals))), evals)
14 show()
15 sums = cumsum(evals)
16 m = max(sums)
17 c = 0
18 while sums[c] < 0.90 * m:
19     c += 1
20 print("To cover 90% of the variance, pick top ", c,
      " eigenvectors")
21 while sums[c] < 0.95 * m:
22     c += 1
23 print("To cover 95% of the variance, pick top ", c,
      " eigenvectors")
24
25 # QPCA 3
26 util.drawDigits(Z.T[:50, :], arange(50))
27 show()

```



- 2 To cover 90% of the variance, pick top 81 eigenvectors
- 3 To cover 95% of the variance, pick top 135 eigenvectors
- 4
- 5 Process finished with exit code 0

- o **Qpca3 (5%):** *Do these look like digits? Should they? Why or why not? (Include the plot in your write-up.) (Make sure you have got rid of the imaginary part in pca.):*



- These do not look like digits. They do not have to look like digits for the same reason that a horizontal line and a vertical line don't look like many 2-dimensional shapes.

- Soft-max Regression

- Qsr1 (10%) (1) Show that the probabilities sum to 1.

Given:

$$P[y = i] = \frac{e^{\vec{w}_i \cdot \vec{x}}}{\sum_j e^{\vec{w}_j \cdot \vec{x}}}$$

Sum over all classes i :

$$\sum_i P[y = i] = \sum_i \frac{e^{\vec{w}_i \cdot \vec{x}}}{\sum_j e^{\vec{w}_j \cdot \vec{x}}} = \frac{\sum_i e^{\vec{w}_i \cdot \vec{x}}}{\sum_j e^{\vec{w}_j \cdot \vec{x}}} = \frac{\sum_i e^{\vec{w}_i \cdot \vec{x}}}{\sum_i e^{\vec{w}_i \cdot \vec{x}}} = 1$$

Hence, the probabilities sum to 1.

- (2) What are the dimensions of W ? X ? WX ?

- W : Number of classes * number of features
- X : Number of features * number of examples
- WX : Number of classes * number of examples

- Qsr2 (15%) (1) Complete the implementation of the cost function.

```
### YOUR CODE HERE ###
probabilities = np.exp(W_X)/np.sum(np.exp(W_X), axis=0)
cost = -np.sum(indicator * np.log(probabilities)) / N
gradient = -np.dot((indicator - probabilities), X.T) / N
### YOUR CODE HERE ###
```

- (2) Complete the implementation of the predict function.

```
### YOUR CODE HERE ###
Probabilities = np.exp(W_X)/np.sum(np.exp(W_X), axis=0)
predicted_classes = np.argmax(probabilities, axis=0)
### YOUR CODE (ENDS) HERE ###
```

- Qsr3 (10%) In the cost function, we see the line

```
W_X = W_X - np.max(W_X)
```

This means that each entry is reduced by the largest entry in the matrix.

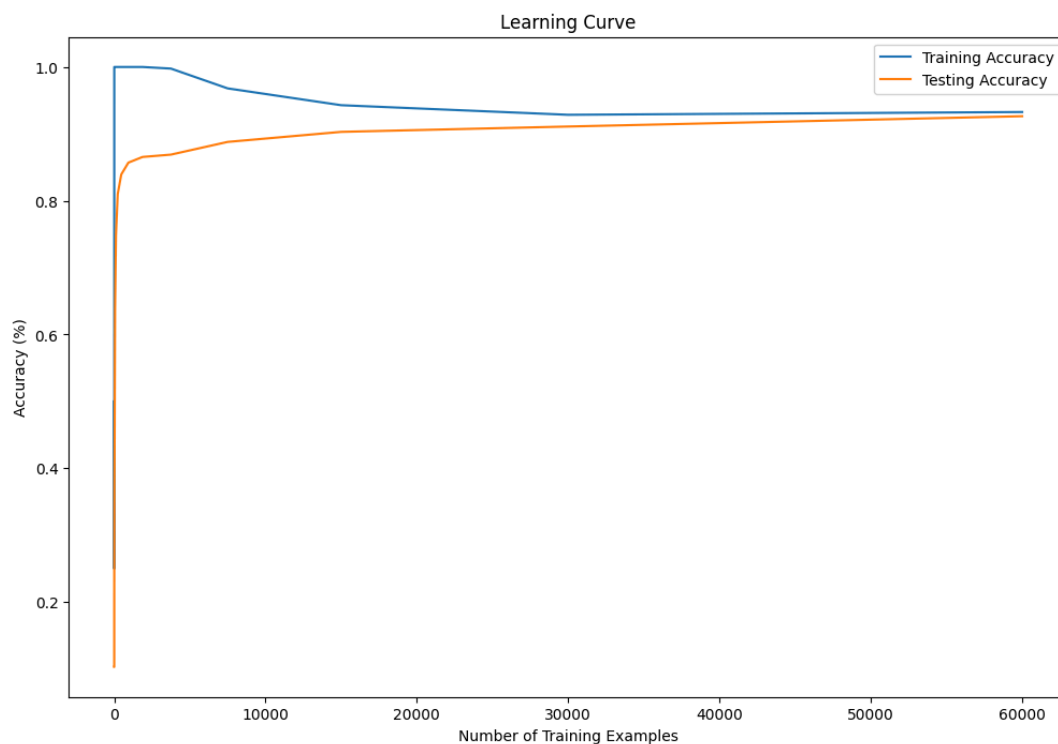
(1) Show that this does not affect the predicted probabilities.

$$\frac{e^{W_{x'_{ij}}}}{\sum_k e^{W_{x'_{ik}}}} = \frac{e^{W_{x_{ij}} - c}}{\sum_k e^{W_{x_{ik}} - c}} = \frac{e^{W_{x_{ij}}} e^{-c}}{\sum_k e^{W_{x_{ik}}} e^{-c}} = \frac{e^{W_{x_{ij}}} (e^{-c})}{(\sum_k e^{W_{x_{ik}}}) (e^{-c})} = \frac{e^{W_{x_{ij}}}}{\sum_k e^{W_{x_{ik}}}}$$

Hence, subtracting the same value from all entries in the matrix would not change the probabilities.

(2) Why might this be an optimization over using W_X ? Justify your answer.

- It allows us to work with smaller numbers which increases precision and reduces the probability of overflows.
- o **Qsr4 (10%)** Use the `learningCurve` function in `runClassifier.py` to plot the accuracy of the classifier as a function of the number of examples seen. Include the plot in your write-up. Do you observe any overfitting or underfitting? Discuss and explain what you observe.



- It seems that training accuracy was higher than test accuracy in the first 10,000 examples and then gradually decreased between them. There is an overfitting in the first stage of a few thousands (maybe 6,000) examples since the training accuracy was almost 100% while the testing accuracy was around 80%. In the long run, there is no overfitting or underfitting as the training and test accuracies are converging to the same value.

- NN

- **Qnn1.1 Implement squared loss cost functions (TODO 0 & TODO 1):**

- Done in nn.py.

- **Qnn1.2 Compute the gradients (TODO 2 & TODO 3):**

- Done in nn.py.
 - Loss goes down as desired.

```

2 activation:Relu
3 loss function:SquaredLoss
4 Layer 1 w:(128, 784)    b:(128, 1)
5 Layer 2 w:(128, 128)    b:(128, 1)
6 Layer 3 w:(10, 128) b:(10, 1)
7 #Samples 6400    loss:0.52179    dev_acc:0.56880
8 #Samples 12800   loss:0.34521    dev_acc:0.69460
9 #Samples 19200   loss:0.29964    dev_acc:0.75430
10 #Samples 25600   loss:0.26739    dev_acc:0.78910
11 #Samples 32000   loss:0.24700    dev_acc:0.81140
12 #Samples 38400   loss:0.23626    dev_acc:0.82970
13 #Samples 44800   loss:0.22305    dev_acc:0.84010
14 #Samples 51200   loss:0.21240    dev_acc:0.84680
15 #Samples 57600   loss:0.20207    dev_acc:0.85540
16
17 Process finished with exit code 0

```

- **Qnn1.3 Run in epochs:**

- Final accuracy on dev set is 95%.

```

2 activation:Relu
3 loss function:SquaredLoss
4 Layer 1 w:(256, 784)    b:(256, 1)
5 Layer 2 w:(256, 256)    b:(256, 1)
6 Layer 3 w:(10, 256) b:(10, 1)
7 Epoch 1/20    loss:0.24041    dev_acc:0.82900
8 Epoch 2/20    loss:0.19728    dev_acc:0.87740
9 Epoch 3/20    loss:0.16214    dev_acc:0.89590
10 Epoch 4/20    loss:0.14154    dev_acc:0.90750
11 Epoch 5/20    loss:0.15564    dev_acc:0.91470
12 Epoch 6/20    loss:0.12863    dev_acc:0.91960
13 Epoch 7/20    loss:0.12619    dev_acc:0.92520
14 Epoch 8/20    loss:0.11157    dev_acc:0.92940
15 Epoch 9/20    loss:0.12092    dev_acc:0.93290
16 Epoch 10/20   loss:0.11616    dev_acc:0.93510
17 Epoch 11/20   loss:0.10735    dev_acc:0.93800
18 Epoch 12/20   loss:0.08694    dev_acc:0.93960
19 Epoch 13/20   loss:0.09704    dev_acc:0.94050
20 Epoch 14/20   loss:0.10033    dev_acc:0.94260
21 Epoch 15/20   loss:0.10209    dev_acc:0.94420
22 Epoch 16/20   loss:0.08914    dev_acc:0.94450
23 Epoch 17/20   loss:0.07554    dev_acc:0.94630
24 Epoch 18/20   loss:0.07594    dev_acc:0.94870
25 Epoch 19/20   loss:0.09645    dev_acc:0.94920
26 Epoch 20/20   loss:0.08489    dev_acc:0.95000
27
28 Process finished with exit code 0

```

- o *Qnn1.4 (No implementation needed for this question). When initializing the weight matrix, in some cases it may be appropriate to initialize the entries as small random numbers rather than all zeros. Give one reason why this may be a good idea.*
 - Assume the first hidden layer of a neural network is fully connected and has more than one node and biases are initialized to 0. If the weight matrix is initialized with identical values, zeros for instance, nodes in the same layer would learn identically, so a single-node layer would have learnt just as good more efficiently, so that multi-node layer wasted computational resources for nothing. Initialize the weight matrix with different values instead, one way to do such is by randomization. With difference fixed, two positive inputs to an activation function vary more when their values are small for most of the common activation functions, for example for tanh, so it is better to randomize on small values to increase variance so that the diversity of what different nodes learn increases.

- Bonus NN
 - o Choose one of the following directions (outside research may be required) for further exploration (Feel free to copy nn.py, utils.py and run_nn.py as a starting point. **Make sure** that your code for Qnn2 is separated from your code for Qnn1): (1) Do dimension reduction with PCA. Try with different dimensions. Can you observe the trade-off in time and acc? Plot training time v.s. dimension, testing time v.s dimension and acc v.s. dimension. Visualize the principal components. (2) Improve your results with ensemble methods. Describe your implementation. Can you observe improved performance compared with that of Q4? Why? (http://ciml.info/dl/v0_99/ciml-v0_99-ch13.pdf) (3) Implement a new optimizer (By implementing a different self.update for the NN class). Compare with the original SGD optimizer. You can read about the optimizers in (<http://ruder.io/deep-learning-optimization-2017/>). Does this new method take less number of samples to converge? Does this new method take less time to converge?
 - o **Qnn2.1** Explain what you did and what you found. Comment the code so that it is easy to follow. Support your results with plots and numbers. Provide the implementation so we can replicate your results.

▪